

PikoGPT Challenge Fact Sheet V1.1

Natural Language Processing with Large Language Models

The Concept: Startup Simulation

We are simulating an early-stage startup phase. You are the founders; we are the investors.

- **The Challenge:** Go from zero to a fully trained LLM with a chat frontend in 12 weeks.
- **Why?** LLMs are transforming the tech landscape and driving the startup ecosystem. Building them yourself from scratch provides the best understanding!
- **Support:** We provide mentoring and **GPU Resources** for final training runs. Prof. Handschuh and TAs acts as Lead Investor, making final “investment” decisions (grading).

1 Organization

- **Professor:** Prof. Siegfried Handschuh
- **Teaching Assistants:** Götz-Henrik Wiegand & Lorena Raichle
- **Communication:** Microsoft Teams or Email

Group Formation:

- Form groups of **4 students** (if you can't find a group, we will assist).
- Choose a **Startup Name**.
- Plan **monthly milestones** until the end of the semester.
- Submit your group members, startup name and the milestones to us today.
- Start with data preprocessing and EDA

Group Plan/Phases:

- Your startup consists of 4 students.
- After pretraining for post-training, you split up into subgroups of 2 students (2x2).
- After post-training, you merge your results together in one tech report and one Poster
- All 4 people work on one codebase with clear separation for the post-training separation.

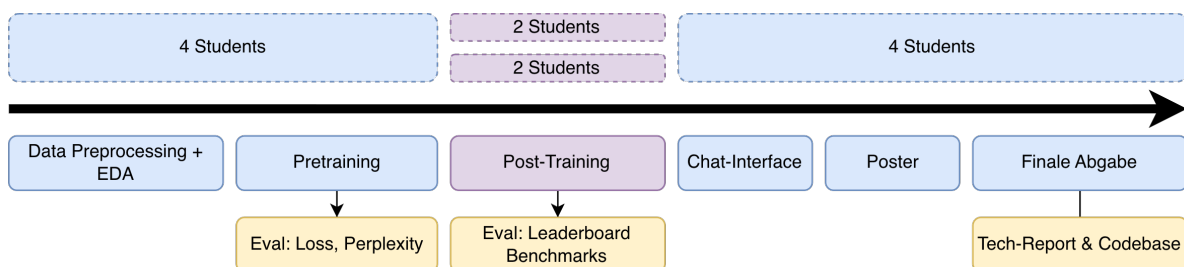


Figure 1: Group Phases

2 Roadmap

The exercise sessions are your dedicated **Startup Lab**. After a short "Code Input" (15 min) connecting theory to practice, the rest of the session is reserved for **onsite team work** and assistance from the TAs.

	Date	Room	Exercise Content	Milestones
1	17.02	Säntis - 1.OG	Motivation & Challenge Intro	Startup Registration
2	24.02	Säntis - 1.OG		
3	03.03	Hilti Inn. R1 - 1.OG		
4	10.03	Winterg. - 1.OG		
5	17.03	Hilti Inn. R2 - 1.OG		
6	24.03	Rosenb. Rotm. - EG		
			— Semester Break —	
			— Semester Break —	
	13.04			Slide submission via Canvas
7	14.04	Vadian Gallus - EG		Midterm Presentation
8	21.04	Vadian Gallus - EG		
9	28.04	Bodensee - 1.OG		
10	05.05	C, 61 61-070		
	11.05			Poster submission via Canvas
11	12.05	Hilti Inn. R2 - 1.OG		Demo & Poster
12	19.05	Hilti Inn. R1 - 1.OG	Pseudo-Conference	
	02.06		Project Submission	
				Code + Report Submission

3 Grading:

The final grade of the course is separated in the following percentages:

- 10% Active participation in lecture and exercise
- 65% Project
 - Repository: Code, Documentation, Readme
 - Benchmarks: Leaderboard Results, Pre-Training Evaluations (Loss, Perplexity)
 - Tech Report
 - Project Work and Participation
 - Individual Contributions Statement via Canvas
- 5% Midterm presentation
- 20% Pseudo Conference
 - Poster
 - Demo Session
 - Conference participation (poster discussions etc.)

4 The Project Task: build a LLM from scratch

If you have any questions or uncertainties, please contact the teaching assistants directly via MS Teams or email.

Any changes or deviations from this fact sheet will be announced in the lecture or exercise.

4.1 PikoGPT Template

You are welcome to use the provided template project:

https://github.com/unisg-ics-dsnlp/PikoGPT_Template

4.2 Technical Constraints (The Rules)

Hard Constraints & Requirements

- **Pretraining Data:** Provided OpenWebText subset.
- **Tokenizer:** GPT-2 tokenizer

```
from transformers import GPT2TokenizerFast
tokenizer = transformers.GPT2TokenizerFast.from_pretrained("gpt2")
```
- **Architecture:** You are encouraged to design your own Architecture within the following constraints!
 - Decoder-Only - No MoE
 - Context size: 1024
 - Max Decoder-Layers: 24
 - Model Size: Max. 40M parameters.
- **Compute Budget:** 2x 24h on 8xV100 (or similar)
- **Tech Stack:** Python & PyTorch. High-level frameworks for the training are forbidden for the main parts of the training and repository. **If you are unsure, ask the TAs.**
- **Forbidden:** using external checkpoints, knowledge distillation, etc.

4.3 Evaluation & Benchmarking

Your "Startup" will be evaluated based on three pillars:

1. General Model Quality:

- Loss and Perplexity Wikitext103 Test Dataset (<https://huggingface.co/datasets/Salesforce/wikitext/viewer/wikitext-103-raw-v1/test>)
- Loss and Perplexity on a given Test-Set from the OpenWebText (Download the test split here: <https://drive.switch.ch/index.php/s/6TLGQFEIkAPJ72K>)

- Important: **You need to make sure, that the Test Datasets are not part of the training in any form!**

2. Leaderboard(Post-Training evaluation):

- **LAMBADA**: Next-word prediction in context.
- **HellaSwag** & **Winogrande**: Multiple Choice / Commonsense reasoning.
- **OpenBookQA**: General knowledge / "Reasoning-light".
- And more hidden benchmarks. (This should give you a Incentive to build as good models as you can and not only train on the known benchmarks.)

3. Systems Engineering:

- **Reproducibility**: 1-line run script, deterministic seeds/configs, well defined stages, uv package management etc.
- **Efficiency**: How well is everything implemented.
- **General Code Standards**: Clean Code, Documentation, etc.

4.4 Leaderboard: Implementation Requirements

Your model will be evaluated using an automated **benchmark runner**. The runner treats your project as a **black box** and interacts with it **only via the command line**.

Key Idea

- We do **not** import your model code. We clone your repository and run a evaluation script.
- We run your model via `python main.py -stage inference`.
- Each benchmark example corresponds to **one inference call**.
- All students are evaluated under **identical conditions**.

Inference Contract (Mandatory)

The inference stage needs to be deterministic (Seed, Temperature = 0, etc.).

Your repository **must** support the following command:

```
python main.py --stage inference \
  --checkpoint CKPT.pt \
  --prompt "..." \
  --max-tokens N \
  --temperature 0 \
  --device auto \
  --leaderboard \
  --seed 0
```

This is the **only interface** used for evaluation.

Required Behavior

- `-stage inference` loads the model and runs text generation
- `-checkpoint` specifies the trained model checkpoint

- `-prompt` is the input text
- `-max-tokens` limits the number of generated tokens
- `-temperature 0` must perform **greedy decoding (argmax)**
- `-device auto` selects the best GPU device (CUDA>MPS>CPU)
- `-seed 0` ensures deterministic behavior

Leaderboard Mode

When the flag `-leaderboard` is set:

- Your program must print **only the generated text** to stdout
- **No logging, banners, or explanations** are allowed

Correct example output:

B

Incorrect output:

```
Loading checkpoint...  
The answer is B
```

Any additional output will cause the example to be marked as invalid.

Benchmarks

Your model will be evaluated on the following public benchmarks:

Benchmark	Task Type	Metric
HellaSwag	Multiple choice (A–D)	Accuracy
WinoGrande	Binary choice (A/B)	Accuracy
OpenBookQA	Multiple choice (A–D)	Accuracy
LAMBADA	Next-word prediction	Accuracy

We also test on some additional hidden benchmarks.

Important Notes

- You can hand in your code + model checkpoint for evaluation multiple times. But make sure you are early enough to not miss an evaluation before the final pseudo conference.
- Leaderboard results are mandatory for the Tech-Report.
- You may structure your code freely, only the inference contract is mandatory!
- If the command fails or output is malformed, the example is invalid

4.5 Demo Interface: Requirements

Key Idea: To showcase the final models and checkpoints, we want you to implement a simple chat interface. This will be part of the pseudo-conference demo-session at the end. We will try out different prompts to compare the different models and showcase the different models on some examples.

Important: The chat-interface is meant as a short demo and inference frontend. You are allowed to use high-level frameworks like Gradio etc.

We don't want you to spend much time in the development of the chat interface. It should just work for demonstration.

Grading: The chat-interface will only be graded on the basic usability and base requirements.

Requirements

- Some sort of GUI (Browser based, Standalone, Terminal-User-Interface, etc.)
- User text input box
- Simple context chat (new prompt = current conversation (context) + user text input)
- Button for new conversation (clears context)
- Option to load different model Checkpoints